UNIVERSITY OF THE PEOPLE

Improving IT Helpdesk Efficiency Through Ticket Categorisation and Workflow Optimisation

Initial Project Report

A Capstone Project Report of MSIT 5910

Submitted by:

Saikat Sunil Ganguly

For the partial fulfilment of the requirements for the degree of

Master of Science in Information Technology

Supervised by:

Dr. Stella Whyte

Department of CS & MSIT

University of the People, Pasadena, CA 91101, United States

October 2026

UNIVERSITY OF THE PEOPLE

ABSTRACT

This capstone examines how IT helpdesk efficiency can be improved through clearer ticket categorisation and a structured triage workflow. Inconsistent classification, vague ticket intake and manual routing delay resolution, cause reassignment and reduce operational visibility. The project first designed a tool-agnostic workflow, grounded in five peer-reviewed studies and three standards, with six modules and a security and governance layer. It then built a minimal working prototype of the core: a Flask web application with rule-based categorisation, role-based access control that masks security-sensitive tickets, and an audit log. The prototype implements three of the six modules. Of the seven functional requirements, six are partly implemented and one, reporting, is not built. It was evaluated on a Windows 11 machine. It categorised 49 of 50 independently written test tickets correctly (98.0%), with a mean response time of 15.3 ms for a single user. A suite of 55 automated tests across unit, integration, system and acceptance levels reached 98% statement coverage. Its first run exposed two defects, empty tickets being stored and an unreliable queue order, which were fixed and re-tested, and three known limitations are documented. A stability test with ten simultaneous users lost no records but slowed noticeably, the slowest request taking 4.7 seconds. The report also sets out a deployment plan, a maintenance plan and a risk register. The results support the view that a simple, explainable rule-based design can be effective and auditable for a constrained category set, and they show clearly what the prototype does not yet do.

UNIVERSITY OF THE PEOPLE

ACKNOWLEDGEMENT

I would like to thank Dr. Stella Whyte for her guidance and feedback throughout this capstone project, from the initial design through the prototype, testing and reporting stages, and the Department of Computer Science and MSIT at the University of the People for the structure and support provided across this course.

UNIVERSITY OF THE PEOPLE

LIST OF TABLES

Table 1. Module Specifications....11
Table 2. Tools and Technologies Used....14
Table 3. Evaluation Objectives, Measures, Evidence and Interpretation....16
Table 4. Functional Requirements and Implementation Status....19
Table 5. Evaluation Metrics and Results....24
Table 6. Categorisation Accuracy by Category....26
Table 7. Test Results by Level....27
Table 8. Defects and Known Limitations Found by Testing....29
Table 9. Results Against the Objectives....30
Table 10. Maintenance Risks and Responses....37
Table 11. Functional Requirements....45
Table 12. Non-Functional Requirements....45
Table 13. Configuration and Setup Requirements....47
Table 14. Proposed Maintenance Schedule....47
Table 15. Test Identifiers and Objectives....47
Table 16. Tagged Releases....48

UNIVERSITY OF THE PEOPLE

LIST OF FIGURES

Figure 1. Conceptual architecture of the proposed helpdesk workflow....11
Figure 2. The login page....20
Figure 3. Result for a password-reset ticket....20
Figure 4. Result for a hardware ticket....21
Figure 5. Ticket queue as the agent role....21
Figure 6. Ticket queue as the identity_management role....21
Figure 7. The /submit route....22
Figure 8. The categorisation function....23
Figure 9. RBAC enforcement logic....23
Figure 10. Database module excerpts....24
Figure 11. Accuracy evaluation output (Windows 11)....25
Figure 12. End-to-end latency measurement (Windows 11)....25
Figure 13. Accuracy by category and latency distribution....26
Figure 14. First test run, with three failing system tests....28
Figure 15. Re-run after the fixes....28
Figure 16. Stability test: 300 submissions from 10 concurrent users....29
Figure 17. Message shown for a whitespace-only ticket....30
Figure 18. Release v1.2.0 on GitHub....49

UNIVERSITY OF THE PEOPLE

SYMBOLS AND ABBREVIATIONS

Abbreviation	Meaning
AT, IT, KL, ST	Acceptance, integration, known-limitation and system test identifiers (for example AT-04)
CI	Continuous integration
CSRF	Cross-site request forgery
FR / NFR	Functional requirement / Non-functional requirement
GDPR	General Data Protection Regulation
HTTP	Hypertext Transfer Protocol
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
IT	Information technology
ITIL	Information Technology Infrastructure Library
MFA	Multi-factor authentication
ms, p95	Millisecond; 95th percentile (95% of requests were faster)
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
SQL, SQLite	Structured Query Language; a file-based relational database engine
TLS	Transport Layer Security
UI	User interface
WSGI	Web Server Gateway Interface

UNIVERSITY OF THE PEOPLE

CONTENTS

ABSTRACT.....i
ACKNOWLEDGEMENT.....ii
LIST OF TABLES.....iii
LIST OF FIGURES.....iv
SYMBOLS AND ABBREVIATIONS.....v
1. INTRODUCTION.....1
1.1 Background of the Study.....1
1.2 Problem Statement.....1
1.3 Objectives of the Study.....3
1.4 Scope and Limitations.....3
1.5 Report Organization.....4
2. LITERATURE REVIEW.....6
2.1 Overview of Related Work.....6
2.2 Comparative Analysis of Existing Systems.....8
2.3 Research Gap Identification.....8
2.4 Testing, Maintenance and Secure Operation.....9
3. SYSTEM DESIGN AND METHODOLOGY.....10
3.1 System Architecture.....10
3.2 Methodologies and Tools Used.....12
3.3 Data Flow and System Components.....14
3.4 Design Diagrams and Evaluation Framework.....16
4. RESULTS AND EVALUATION.....18
4.1 Implementation Overview.....18
4.2 Output and Screenshots.....20
4.3 Evaluation Metrics.....24
4.4 System Testing.....27
4.5 Summary of Results Against the Objectives.....30
5. DISCUSSION.....31
5.1 Interpretation of Results.....31
5.2 Comparison with Literature Findings.....32
5.3 Implications and Limitations.....34
5.4 Deployment, Configuration and Maintenance.....36
5.5 Threats to Validity.....37
6. CONCLUSION AND FUTURE WORK.....39

6.1 Summary of Findings....39
6.2 Contributions....40
6.3 Lessons Learned and Challenges....40
6.4 Future Work....41
REFERENCES....42
APPENDICES....45
Appendix A. Functional and Non-Functional Requirements....45
Appendix B. Deployment, Configuration and Maintenance Plan....46
Appendix C. Test Catalogue....47
Appendix D. Repository and Release History....48

UNIVERSITY OF THE PEOPLE

1. INTRODUCTION

1.1 Background of the Study

IT helpdesks are a primary support channel for employees who experience technical incidents or need routine IT services. Their effectiveness depends not only on technical capability but also on how consistently tickets are captured, categorised, prioritised, routed, and monitored. Where ticket intake is vague and triage depends heavily on individual interpretation, support teams may spend additional time clarifying requests, reassigning tickets, or responding inconsistently to similar incidents. This is a well-documented operational problem: as helpdesk ticket volumes grow, the manual effort required to interpret and route free-text requests grows with them, and inconsistent handling becomes more visible in delayed resolutions and repeated escalations.

This problem is significant beyond any single organisation because helpdesk operations are a near-universal function across IT-dependent businesses. A single misrouted ticket may cost only a few minutes of reassignment, but at hundreds or thousands of tickets per month even a modest drop in categorisation accuracy means measurably more agent workload, longer user waits and noisier reports. That scaling effect is why the literature reviewed in Chapter 2 treats categorisation accuracy at first contact as a meaningful design target and not as a minor implementation detail.

1.2 Problem Statement

Many IT helpdesks rely on user-submitted free text and manual triage processes that vary by agent, queue, or workload. As a result, tickets may be assigned to an inappropriate category,

routed to the wrong team, or delayed while support staff request clarification. Reassignment creates extra work for users and support teams and can reduce the reliability of operational reporting. This pattern is consistent with what Al-Hawari and Barham (2021) describe as the cost of inaccurate first-contact categorisation: tickets that are miscategorised at submission tend to accumulate delay and rework as they move through subsequent handling stages, rather than the error being isolated to a single step.

The problem affects several stakeholders in distinct ways. End users experience longer disruption and reduced confidence when they must clarify or resubmit requests, particularly for time-sensitive issues such as access problems that block their ability to work. Helpdesk agents spend time interpreting vague descriptions and transferring tickets between queues, which reduces the time available for actual resolution work. IT managers have less reliable information for measuring service performance or justifying resourcing decisions when category and priority data cannot be trusted. Engineering and specialist teams can be interrupted when tickets are misrouted into their queues, reducing capacity for planned project work.

A representative illustration clarifies the intended intervention point. A user who submits a ticket saying only “my account is not working” may be describing an access issue, an application error or a security concern, and different agents may categorise it differently depending on judgement or workload. If it is treated as a routine access issue it may wait in a standard queue for hours; if the cause is a compromised account, that delay is a security exposure and not a minor inconvenience. Short, vague descriptions of this kind are common, because users describe the symptom they see and cannot be expected to diagnose its cause. The guiding research question for this capstone is: How can a simplified ticket categorisation model and structured triage

workflow improve routing consistency and reduce resolution delays in IT helpdesk environments?

1.3 Objectives of the Study

This capstone is guided by five objectives:

1. To identify a set of standard ticket categories and priority criteria, grounded in the literature reviewed in Chapter 2, that reduce ambiguity at ticket intake without requiring specialist technical knowledge from the end user.

2. To design a structured triage and escalation workflow and system architecture, documented in Chapter 3, that separates routine handling from security-sensitive escalation and applies security and governance controls throughout.

3. To implement a minimal working prototype of the core modules (ticket submission, rule-based categorisation, and role-based access control with an audit trail), described in Chapter 4.

4. To evaluate and test the prototype for categorisation accuracy, response time, usability, functional correctness and stability, using the framework in Table 3 and the testing described in Section 4.4.

5. To plan the deployment, configuration and long-term maintenance of the system, and to document the version-controlled artifacts that support it (Section 5.4 and the appendices).

1.4 Scope and Limitations

The project began as a process-focused design study and was extended, with the supervisor's guidance, to include a deliberately small software prototype. It does not propose a replacement

helpdesk platform, machine-learning automation or integration with proprietary systems; it provides a workflow blueprint that can be adapted within an existing helpdesk environment, together with a working proof of concept of its core logic. Identifiable organisational ticket data are excluded, because the project has no ethical or contractual access to a live production system.

This has consequences for what the report can claim. First, the prototype implements three of the six designed modules (ticket submission, categorisation, and the security and governance layer); the triage, assignment and escalation, and reporting modules remain at the design stage, so the results describe the implemented logic and not the full workflow. Second, evaluation uses 50 hand-written hypothetical tickets, a local test environment and a single evaluator, not production outcomes. Third, the categorisation model is intentionally rule-based and human-readable rather than trained; this trade-off was accepted because the reviewed literature (Revina et al., 2020; Wahba et al., 2020) indicates that simpler approaches are frequently competitive with more complex ones in this domain. Fourth, the design assumes existing identity-management infrastructure, and the prototype's login accepts a username without a password, so it is suitable only for demonstration. These limits define what this report can responsibly conclude, and they shape the future work in Chapter 6.

1.5 Report Organization

Chapter 2 reviews the literature on ticket classification, service management, testing, maintenance and secure operation, and identifies the research gap. Chapter 3 presents the system design and methodology, including the architecture, tools and algorithm. Chapter 4 reports the implementation, the evaluation results and the system testing. Chapter 5 interprets the results, compares them with the literature, discusses limitations, summarises the deployment,

configuration and maintenance plans, and considers threats to validity. Chapter 6 concludes with the contributions, lessons learned and future work. The appendices contain the full requirements, the deployment plan, the test catalogue and the repository history.

2. LITERATURE REVIEW

2.1 Overview of Related Work

Academic research indicates that ticket classification quality has an important influence on routing and support efficiency. Revina et al. (2020) investigated IT ticket classification methods and found that simpler, explainable linguistic features can be effective for classifying support tickets, in some cases outperforming more complex modelling approaches. Their study supports the use of clear, understandable category prompts rather than relying solely on complex technical mechanisms.

Al-Hawari and Barham (2021) developed and evaluated a machine learning based help desk system intended to associate an incoming ticket with the correct service category from the point of submission, with the explicit goal of minimising ticket resolution time and reducing unnecessary reassignment. Their methodology demonstrates that correct first-contact categorisation is a measurable and consequential design target, not only a convenience feature.

A broader view of the field is provided by Fuchs et al. (2022), who conducted a systematic literature review of machine learning applications in support ticket systems. Their review found that automated incident management tools represent the majority of research activity in this area, followed by request escalation prediction and customer sentiment analysis, and identified Random Forest and Support Vector Machine algorithms as consistently strong performers for ticket classification tasks.

Wahba et al. (2020) tested static word-embedding models against a simpler term-frequency representation combined with a linear classifier on a large-scale, real-world dataset of

approximately 1.6 million support tickets spanning 32 categories. They found that the simpler term-frequency approach matched or outperformed the more computationally expensive embedding models while requiring substantially less training time and computational cost, reinforcing the theme established by Revina et al. (2020) that added technical complexity does not automatically translate into better categorisation outcomes.

Jäntti and Cater-Steel (2017) examined proactive management of IT service operations across three organisations. They identified reactive service-desk patterns, inconsistent classification, weak measurement practices, and poor process interfaces as barriers to effective IT service management, supporting a process-improvement approach: service-desk performance can be strengthened through clearer workflows and measurement rather than only through system replacement.

ITIL 4 provides a relevant service-management foundation. Incident management seeks to restore normal service operation as quickly as possible while minimising adverse business impact (AXELOS, 2019). Standardised categorisation and prioritisation help support this objective by enabling incidents to be understood and routed consistently.

Information security and privacy are also relevant because helpdesk tickets can contain personal, operational, or security-sensitive information. ISO/IEC 27001:2022 identifies the importance of managed information-security controls, including access management and accountability (International Organization for Standardization, 2022). The General Data Protection Regulation highlights principles including data minimisation, purpose limitation, and secure processing for personal data (European Union, 2016).

2.2 Comparative Analysis of Existing Systems

The reviewed literature reflects two broad approaches to improving ticket handling. The first is automated classification, exemplified by Al-Hawari and Barham (2021), Fuchs et al. (2022) and Wahba et al. (2020), which applies machine-learning models, from linear classifiers to embedding-based methods, to predict a category or route from ticket text. Such approaches scale to very large ticket volumes, but they require labelled historical data, ongoing model maintenance and technical capacity that many organisations, including this capstone, do not have available.

The second approach is process-level improvement, exemplified by Jäntti and Cater-Steel (2017) and the ITIL 4 incident-management framework (AXELOS, 2019), which improves consistency through clearer workflows, defined categories and measurement. It is cheaper to adopt and does not depend on historical data, but it relies on disciplined human execution. Revina et al. (2020) and Wahba et al. (2020) bridge the two: both found that simple, well-structured, explainable methods can be competitive with more complex ones. That finding informed the decision, set out in Chapter 3, to build a documented rule-based system instead of a trained classifier, a defensible design choice with empirical support.

2.3 Research Gap Identification

A gap remains between the two bodies of research. The classification literature is largely evaluated in terms of model accuracy on historical ticket datasets, and rarely addresses how a non-technical organisation without machine-learning capacity could apply the same underlying insight, namely that clear, structured categorisation improves outcomes, using only workflow design. The service-management literature, in turn, discusses process improvement in general

terms but does not typically translate specific classification findings into a concrete intake and triage specification. This capstone sits at the intersection of the two: it takes the empirical finding that simple, well-structured categorisation is effective (Revina et al., 2020; Wahba et al., 2020) and operationalises it as a documented, tool-agnostic workflow rather than a trained model, informed by the service-desk process weaknesses identified by Jäntti and Cater-Steel (2017).

2.4 Testing, Maintenance and Secure Operation

Because the project moved from design to a working prototype, three further bodies of guidance shaped the later chapters. Testing practice distinguishes white-box techniques, which exercise internal structure, from black-box techniques, which check behaviour from outside (Khan & Khan, 2012). Maintenance guidance divides post-deployment work into corrective, adaptive, perfective and preventive types (ISO/IEC/IEEE, 2022), and continuous integration is reported to work on maintenance projects as well as new ones (Shahin et al., 2017). Security guidance such as the OWASP Foundation (2021) Top 10 names broken access control, security misconfiguration and identification and authentication failures among the most critical web application risks, while SQLite's own documentation states that it allows many simultaneous readers but only one writer at a time (SQLite, n.d.). These sources are applied in Sections 4.4 and 5.4.

3. SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture

The proposed architecture contains six connected components: a ticket submission interface, categorisation module, triage and assessment module, assignment and escalation module, reporting and analytics module, and security and governance layer. Figure 1 illustrates the flow between these components and shows the security and governance layer applying across every stage instead of being confined to a single control point. The architecture is conceptual and can be applied to an existing helpdesk platform. Table 1 summarises each module's inputs, outputs, methods and safeguards, and the functional and non-functional requirements are listed in Appendix A.

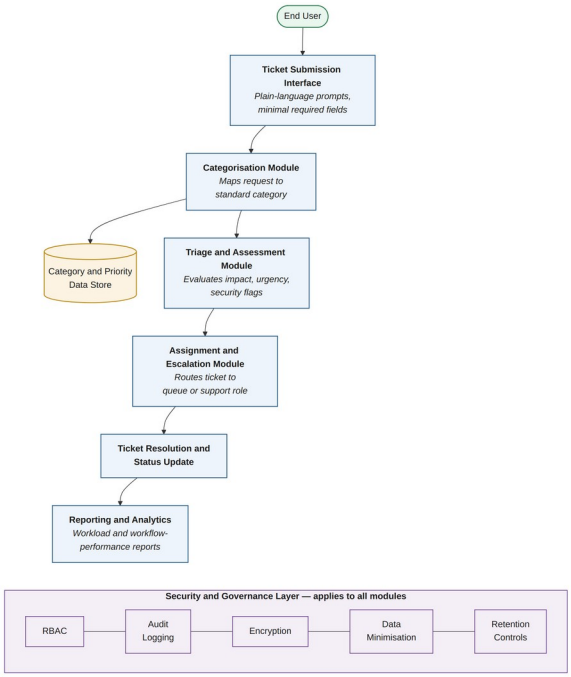


Figure 1. Conceptual architecture of the proposed IT helpdesk categorisation and triage workflow.

Table 1

Module Specifications

Module	Inputs	Outputs	Methods	Main safeguards
Ticket Submission Interface	User issue description, guided responses, optional supporting information	Structured ticket record with initial category and priority cues	Plain-language prompts, minimal required fields, input validation	Consent notice, TLS, data minimisation
Categorisation Module	User responses, category lookup table	Standard category and preliminary priority indicator	Guided classification, lookup values, validation rules	Consistent prompts, prevention of incomplete categorisation
Triage and Assessment Module	Categorised ticket, urgency and impact responses	Routing decision, priority, and escalation indicator	Decision matrix, impact/urgency assessment, security flagging	Immediate handling path for security-sensitive tickets
Assignment and Escalation Module	Routing decision, role and queue rules	Assigned queue or support role, escalation status	Role-based routing, SLA-aligned escalation criteria	RBAC, assignment audit log
Reporting and Analytics Module	Ticket history, category, assignment, status and resolution data	Workload and workflow-performance reports	Reassignment analysis, backlog tracking, resolution-time trends	Aggregated reporting and retention controls

Module	Inputs	Outputs	Methods	Main safeguards
Security and Governance Layer	User, ticket, role and audit information	Access-control and compliance evidence	RBAC, encryption, logging, data minimisation, retention policy	Least privilege, accountability, secure handling

The safeguards listed, particularly role-based access control, least-privilege access and audit logging, are consistent with the access-control principles of ISO/IEC 27001:2022 (International Organization for Standardization, 2022).

3.2 Methodologies and Tools Used

Design methodology

The ticket submission interface uses plain-language prompts and minimal required fields, such as "I cannot access something" or "Something is not working," to help non-technical users describe their issue without technical vocabulary. The categorisation module maps each request to a controlled set of categories using a documented rule set instead of a trained classifier, directly informed by Revina et al.'s (2020) and Wahba et al.'s (2020) findings that simpler, well-structured category schemes are competitive with automated methods. The design named five categories (Access Issues, Application Errors, Hardware Problems, Service Requests and Security Concerns); during implementation these were refined into the five categories described in Section 3.3.

The triage and assessment module applies a documented impact-and-urgency decision matrix, using guiding questions such as "Is the issue preventing the user from working?" to support consistent priority decisions. Security-related or high-impact tickets are flagged for an immediate escalation path, reflecting the ISO/IEC 27001 principle of protecting sensitive information (International Organization for Standardization, 2022). The assignment and escalation module

then uses category, priority and queue rules, supported by role-based access control, to direct the ticket to the right support group. Only the submission, categorisation and security components were implemented in the prototype (Chapter 4).

The architecture diagram was produced with Mermaid, a text-based diagramming tool, so that its source can be version-controlled alongside the documentation. The reporting module uses standard aggregation (counts, rates and elapsed times) instead of predictive modelling, which keeps its logic transparent and auditable and supports the fairness and transparency requirement (NFR5). Across all modules the methodology favours documented, human-readable rules over trained models, so that the design is coherent and auditable end to end.

Development process

Development was iterative. Design and literature work produced the architecture and requirements; a first prototype with a recorded demonstration followed; then unit testing and version-control practice, integration, evaluation and deployment planning, and finally system testing and maintenance planning. Work was tracked in a GitHub repository (Ganguly, 2026), with a main branch for reviewed releases and a development branch for active work, and every milestone was tagged with a semantic version (Preston-Werner, n.d.), from v0.1.0 (first prototype) to v1.2.0 (system testing). Appendix D lists the releases.

Categorisation algorithm

The implemented algorithm is deliberately simple. The ticket text is converted to lower case once; for each of the five stored categories the algorithm counts how many of its keywords occur in the text; the category with the strictly highest count wins, bringing its default priority and security-sensitive flag; and if no keyword matches, the ticket is routed to “Uncategorized - Needs

Human Review" and not guessed. Access control is enforced on the server: when the queue is requested, the description of every security-sensitive ticket is replaced by a fixed message unless the session role is identity_management. Every login, logout, ticket submission and queue view is written to an audit log. Section 4.2 shows the code.

Tools and technologies

Table 2

Tools and Technologies Used

Tool	Purpose
Python 3.14 and Flask	Web application, routing and page templates
SQLite	File-based relational database for categories, tickets, users and the audit log
pytest and pytest-cov	Automated tests and statement coverage
requests and matplotlib	HTTP load and stability scripts; result charts
Git and GitHub	Version control, tagged releases and issue tracking
Mermaid	Text-based architecture diagram
Docker and gunicorn (planned)	Containerised deployment and production web server (Appendix B)

3.3 Data Flow and System Components

Data flows from the end user, through the ticket submission interface, into the categorisation module, which writes a standard category and preliminary priority indicator to a shared category and priority data store. The triage and assessment module reads this record, applies the impact-and-urgency matrix, and passes a routing decision to the assignment and escalation module, which assigns the ticket to a queue or role and updates its status. Resolution and status updates then feed the reporting and analytics module, which aggregates workload and workflow-performance data such as reassignment rate, time to initial route, time to resolution, and backlog size. Because the project does not use proprietary or identifiable organisational ticket data, this

data model is described at a conceptual level: a ticket record is treated as containing only the fields necessary to support categorisation, triage, assignment, and reporting, deliberately excluding personal identifiers beyond what is needed to contact the requester. This reflects the GDPR data-minimisation principle discussed in Chapter 2 and the least-privilege posture recommended by ISO/IEC 27001 (International Organization for Standardization, 2022): each module receives only the information it needs to perform its function, rather than passing a full user profile through every stage.

As implemented, the data store has four SQLite tables: categories (five seeded categories with keyword lists, a default priority and a security-sensitive flag), tickets, users (two demonstration roles) and audit_log. They correspond to the Category and Priority Data Store, the Ticket Submission Interface output, the role model for access control, and the logging safeguard of the Security and Governance Layer. The five implemented categories are Password / Account Access, Hardware Issue, Software / Application Error, Network Connectivity and Access Request / Permissions. Security concerns are handled as a flag on the Password / Account Access and Access Request / Permissions categories, which triggers masking, and not as a separate category.

A worked example shows how the flow supports the evaluation objectives in Table 3. Returning to the scenario in Section 1.2, “my account is not working,” the prototype matches the keyword “account,” assigns the category Password / Account Access with High priority, and flags the ticket as security-sensitive, so its description is hidden from first-level agents. That flag is how the design's immediate escalation path is realised in the prototype: as an access restriction. Running a representative set of such scenarios provides the basis for the evaluation in Chapter 4.

3.4 Design Diagrams and Evaluation Framework

Figure 1 serves as the primary design diagram, showing the system as a flow of connected modules with a security and governance layer applied throughout. A separate data-flow or entity-relationship diagram was not produced, because the as-built data model is small and is described in Section 3.3. The evaluation framework used throughout Chapter 4 is shown in Table 3.

Table 3

Evaluation Objectives, Measures, Evidence and Interpretation

Evaluation objective	Measure	Evidence	Interpretation
Categorisation clarity	Correct category selection	Scenario outcome compared with expected category	Correct or defensible selection indicates a clear category guide.
Routing consistency	Incorrect route or reassignment points	Initial routing result compared with expected queue	Fewer ambiguous routes indicate stronger routing consistency.
Triage efficiency	Decision steps needed before routing	Workflow walkthrough and step count	Fewer unnecessary decision points indicate reduced triage effort.
Priority handling	Correct priority and escalation decision	Scenario outcome compared with priority matrix	High-impact or security-sensitive cases should receive the documented route.
Operational visibility	Availability of fields for reporting	Requirements and reporting-field review	The design should capture category, priority, assignment, status and resolution information.
Information protection	Coverage of safeguards	Security requirements review	RBAC, audit logging, data minimisation and retention controls should be specified.

The framework was written before implementation, using hypothetical scenarios because the project has no access to production ticket history and identifiable data would raise privacy and consent concerns. In practice the scenarios became a 50-ticket labelled test set (categorisation clarity), the automated tests and browser scenarios (routing, priority handling and information protection), and timed requests (triage efficiency). Operational visibility could not be evaluated,

because the reporting module was not built. Positive results are therefore treated as evidence that the implemented logic works, not as verified organisational gains.

4. RESULTS AND EVALUATION

4.1 Implementation Overview

This chapter reports what was built, tested and measured, not what was planned. It covers the first working prototype of the design in Chapter 3, the unit tests that verified its core logic, the system integration and evaluation that followed, and the system testing added in the final phase.

The implementation covers three of the six modules from the System Architecture (Section 3.1) as a working Flask application: the Ticket Submission Interface, the Categorisation Module, and the Security and Governance Layer (role-based access control and audit logging). The Triage and Assessment, Assignment and Escalation, and Reporting and Analytics modules remain at the design stage; this is a deliberate scope decision, consistent with the capstone's original framing as a process-design project rather than a full software build, and with Dr. Whyte's guidance during the prototype phase that the implementation should stay “as small as possible” while still demonstrating the core logic of the approved design.

Core Algorithm Implementation

The first version of the prototype was validated through a recorded system demonstration. Nineteen white-box unit tests were then written against categorize_ticket() and the database layer, achieving 100% statement coverage on both modules. That testing surfaced a defect: a generic “reset” keyword in the Password / Account Access category matched unrelated hardware tickets such as “reset my monitor settings”. Replacing it with specific phrases fixed the problem, and a regression test now locks the fix in. A related tie-breaking weakness reappears in Section 4.3.

The implemented modules were integrated into one Flask application and run as a real local web server on a Windows 11 machine (Python 3.14.2), then exercised through actual HTTP requests and a real browser and not only through isolated function calls. Every screenshot and measurement in this chapter comes from that environment. Table 4 maps the seven functional requirements (listed in Appendix A) to what was implemented. No requirement is met in full: six are partly implemented and one, reporting (FR6), is not built. FR1, FR4 and FR7 are the three whose implemented parts could be tested against running code, and the acceptance tests cover them. The other four (FR2, FR3, FR5 and FR6) depend on modules not built in this phase and are partly represented through simplified fields or deferred. Stating these gaps precisely means the results that follow describe the categorisation and access-control logic, and should not be read as a performance claim about the full six-module design.

Table 4

Functional Requirements and Implementation Status

ID	Functional Requirement	Implementation Status
FR1	Guided prompts covering at least five standard ticket categories	Partially implemented: tickets are classified automatically into five seeded categories (Figures 3 and 4); guided category-selection prompts are not built
FR2	Structured ticket record with category, impact, urgency, description, timestamp	Partially implemented: category, priority, description and timestamp are recorded; impact and urgency are combined into a single priority value
FR3	Standard triage workflow routing by category, priority, impact, urgency	Partially implemented: routing uses category and priority; impact and urgency are not separately scored
FR4	Flag security-sensitive or high-impact tickets for escalation	Partially implemented: security-sensitive tickets are flagged and masked by role (Figures 5 and 6); a distinct escalation notification is not built
FR5	Record category, priority, assignment, status, resolution for analysis	Partially implemented: category, priority, status and submitter are recorded; agent assignment and resolution outcome are not yet modelled

ID	Functional Requirement	Implementation Status
FR6	Produce reports on reassignment, backlog, resolution time, categorisation quality	Not implemented: the Reporting and Analytics module remains conceptual; this is the biggest scope gap (Section 5.3)
FR7	Enforce role-based access control and maintain an audit trail	Partially implemented: RBAC shown in Figures 5 and 6, and every login, logout, submission and queue view is written to the audit_log table; assignment and escalation changes are not audited because those functions are not built

4.2 Output and Screenshots

Figures 2 to 6 are browser screenshots of the running application, taken while logging in, submitting two different tickets, and viewing the resulting queue under the restricted (agent) and privileged (identity_management) roles.

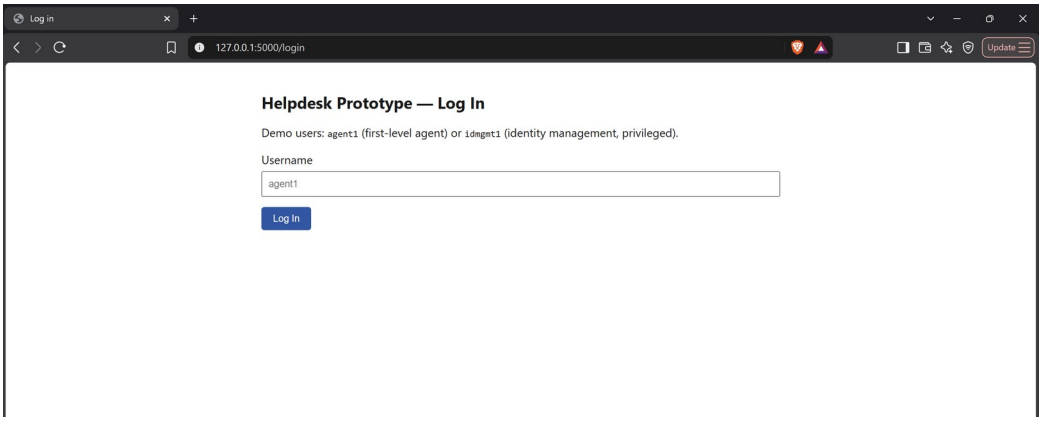


Figure 2. The login page, rendered from the running application.

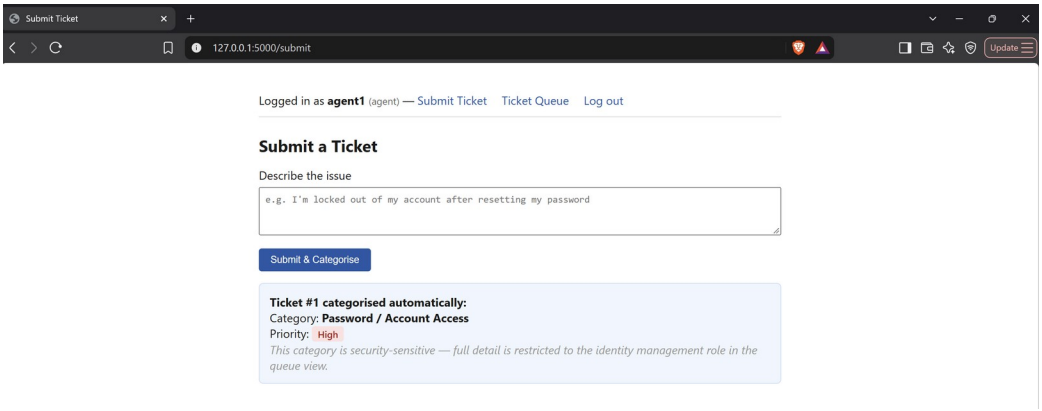


Figure 3. Categorisation result for a password-reset ticket (Ticket #1): Password / Account Access, High priority, flagged security-sensitive.

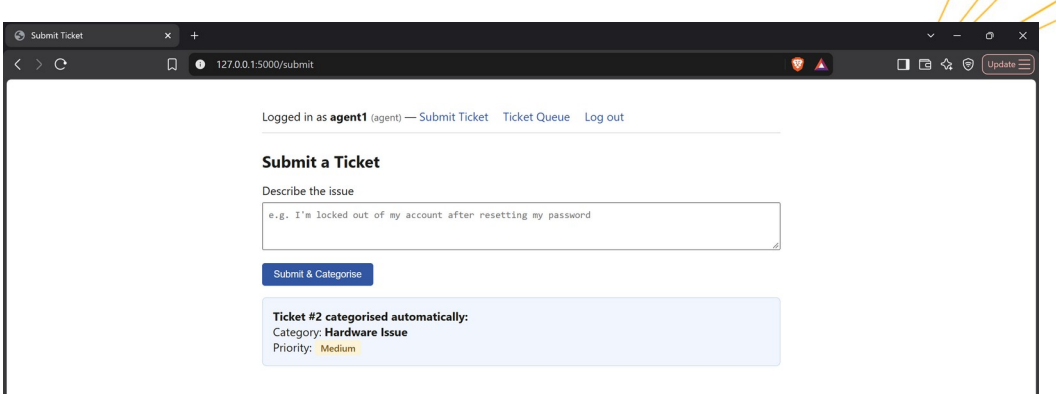


Figure 4. Categorisation result for a hardware ticket (Ticket #2): Hardware Issue, Medium priority, a different outcome driven by the input text alone.

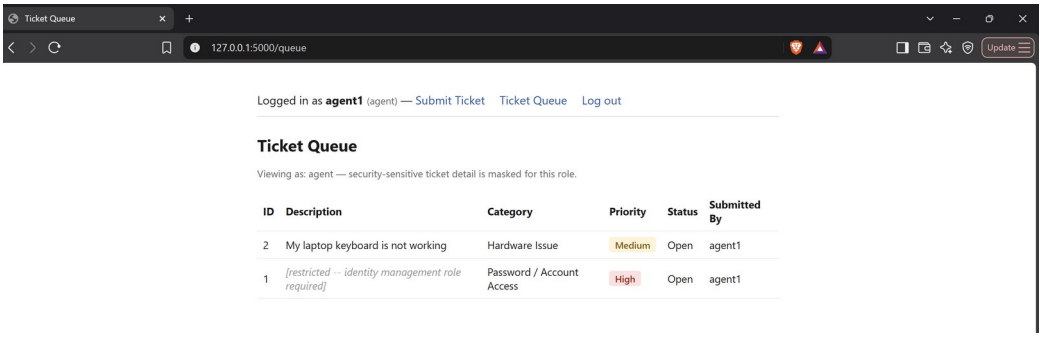


Figure 5. Ticket queue as the agent role: Ticket #1's description is masked because it is security-sensitive, and the newest ticket is listed first.

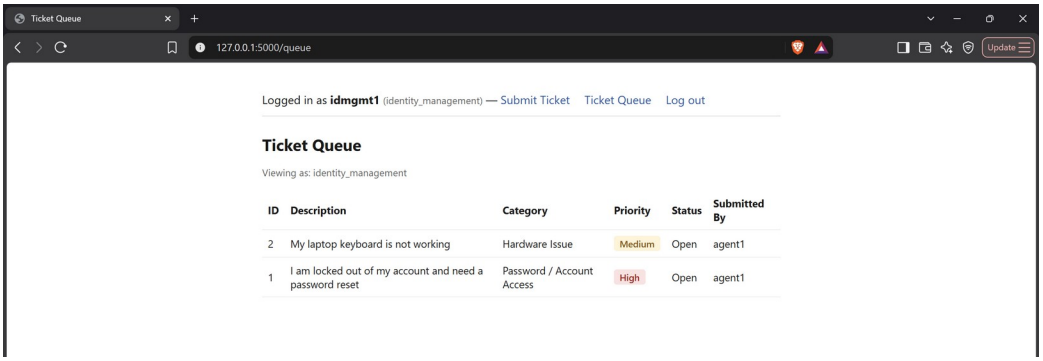


Figure 6. The same queue and data as the identity_management role: the full description is visible, showing that the RBAC control is enforced on the server.

Figure 7 shows the /submit route, where the Ticket Submission Interface, the categorisation module and the database layer meet. It reads the description from the HTTP request, rejects an empty one, calls the categorisation logic, writes the ticket through the database module and records the action in the audit log before returning the result.

```python
@app.route("/submit", methods=["GET", "POST"])
def submit_ticket():
    if "username" not in session:
        return redirect(url_for("login"))

    result = None
    if request.method == "POST":
        description = request.form["description"].strip()
        if not description:
            # Unit 7 fix (found by tests ST-09 and ST-10): empty tickets used to be stored.
            return render_template(
                "submit.html", username=session["username"], role=session["role"],
                result=None, error="Please describe the issue before submitting.",
            ), 400
        category, priority, sensitive = categorize_ticket(description)

        conn = get_connection()
        conn.execute(
            "INSERT INTO tickets (description, category, priority, security_sensitive, submitted_by) "
            "VALUES (?, ?, ?, ?, ?)",
            (description, category, priority, int(sensitive), session["username"]),
        )
        ticket_id = conn.execute("SELECT last_insert_rowid()").fetchone()[0]
        conn.commit()
        conn.close()

        log_action(session["username"], "submit_ticket", ticket_id)
        result = {"category": category, "priority": priority, "sensitive": sensitive, "id": ticket_id}

    return render_template("submit.html", username=session["username"], role=session["role"], result=result)
```

Figure 7. The /submit route (app.py), including the empty-ticket check added after system testing.

Core Module Code

Figure 8 is the complete categorisation function. It lower-cases the description once, compares it against every category's keyword list, keeps the category with the strictly highest match count, and falls through to “Uncategorized - Needs Human Review” if nothing matched. The strict comparison (match_count > best_match_count) produces the tie-breaking behaviour discussed in Sections 4.3 and 5.1: a later category can never displace an earlier one on an equal score.

```python
def categorize_ticket(description: str):
    """
    Match a ticket description against the category keyword table.
    Returns (category_name, priority, security_sensitive) for the best match,
    or a fallback "Uncategorized" result if nothing matches -- this fallback
    is itself part of the fairness design: uncertain tickets get routed to
    human review rather than being forced into a guessed category.
    """
    description_lower = description.lower()

    conn = get_connection()
    categories = conn.execute("SELECT * FROM categories").fetchall()
    conn.close()

    best_match = None
    best_match_count = 0

    for cat in categories:
        keywords = [k.strip() for k in cat["keywords"].split(",")]
        match_count = sum(1 for k in keywords if k in description_lower)
        if match_count > best_match_count:
            best_match_count = match_count
            best_match = cat

    if best_match is None:
        # No keyword matched -- route to human review rather than guess.
        return "Uncategorized - Needs Human Review", "Medium", False

    return best_match["name"], best_match["default_priority"], bool(best_match["security_sensitive"])
```

Figure 8. The categorisation function (categorization.py).

Figure 9 shows the RBAC check in the /queue route, which implements the access-control part of FR7. Every ticket is returned, but a security-sensitive ticket's description is replaced by a fixed message unless the signed-in role is identity_management. Because the check runs on the server, the restricted text never reaches an unprivileged browser, so it cannot be exposed through page-source inspection or ordinary request replay. This is an authorisation control and not an authentication one (Section 5.3).

```python
    # RBAC enforcement: agents see every ticket, but the description of a
    # security-sensitive ticket (e.g. password/account access) is masked
    # unless the viewer is identity_management. This mirrors the
    # "account-recovery details" example from Document 1, Section 4.
    is_privileged = session["role"] == "identity_management"
    visible_tickets = []
    for t in tickets:
        row = dict(t)
        if row["security_sensitive"] and not is_privileged:
            row["description"] = "[restricted -- identity management role required]"
        visible_tickets.append(row)

    return render_template(
```

Figure 9. RBAC enforcement logic (app.py).

Figure 10 shows excerpts of the database module: the connection helper, the audit_log table and the function that records every action. The full set of tables is described in Section 3.3.

```python
def get_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn
    ...
    cur.execute("""
        CREATE TABLE IF NOT EXISTS audit_log (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            action TEXT NOT NULL,
            ticket_id INTEGER,
            timestamp TEXT NOT NULL DEFAULT (datetime('now'))
        )
    """)
    ...
def log_action(username, action, ticket_id=None):
    conn = get_connection()
    conn.execute(
        "INSERT INTO audit_log (username, action, ticket_id) VALUES (?, ?, ?)",
        (username, action, ticket_id),
    )
    conn.commit()
    conn.close()
```

Figure 10. Database module excerpts (models.py).

4.3 Evaluation Metrics

Table 5 summarises the three metrics evaluated against the running system: two quantitative and one qualitative, consistent with the framework in Table 3. The 50 test tickets, the evaluation scripts and the results of this run are available in the project repository (Ganguly, 2026).

Table 5

Evaluation Metrics and Results

Metric	Type	Method	Result
Categorisation accuracy	Quantitative	50-item hand-labelled test set, 10 per category	98.0% (49/50)
POST /submit latency	Quantitative	60 HTTP requests, full stack	Mean 15.29 ms, median 14.85 ms, p95 20.58 ms
Usability (heuristic walkthrough)	Qualitative	Nielsen (1994) heuristics against the running interface	5 of 6 selected heuristics fully met; error prevention partly met

Categorisation accuracy was measured against a 50-item test set of ten realistic ticket descriptions per category, written independently of the keyword lists so that the test would not simply confirm its own assumptions. It operationalises the “Improve categorisation clarity” objective in Table 3, whose success criterion was that a relevant category can be selected without

additional clarification in most scenarios. The result was 98.0% (49 of 50), shown in Figure 11, with one misclassification: “My laptop shows offline even though others are connected” was expected to be Network Connectivity but was categorised as Hardware Issue, because “laptop” and “offline” each match exactly one keyword and ties go to whichever category was seeded first. This is the same tie-breaking behaviour documented in the unit tests, now observed on an independent test item, so it is a systematic property of the design and not a one-off error.

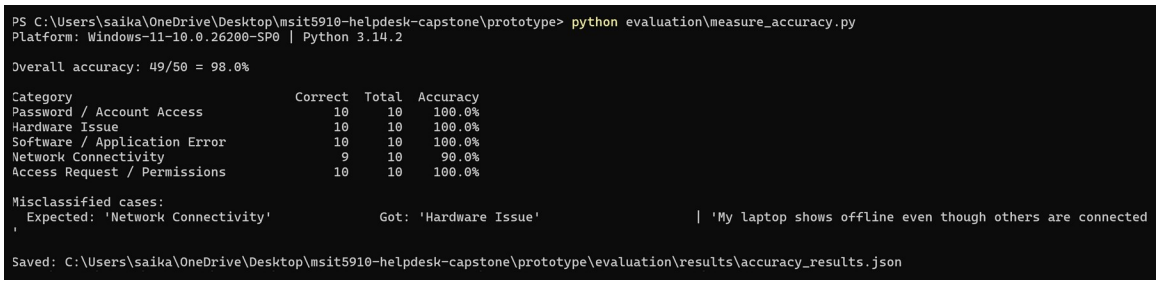


Figure 11. Accuracy evaluation output (Windows 11), including the one misclassification and its per-category breakdown.

Latency was measured as the full round-trip time of 60 POST /submit requests against the live server, covering routing, the categorisation call, the SQLite write and template rendering. It ran on the same Windows 11 machine. The result was a mean of 15.29 ms and a 95th percentile of 20.58 ms, with no request above 24.11 ms (Figure 12), which speaks to the “Improve initial triage efficiency” objective: the automated categorisation step is not a meaningful source of delay at this scale. Figure 13 shows both quantitative results together.

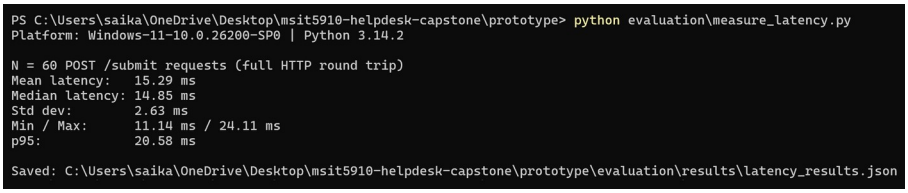


Figure 12. End-to-end latency measurement (Windows 11), n = 60 HTTP requests.

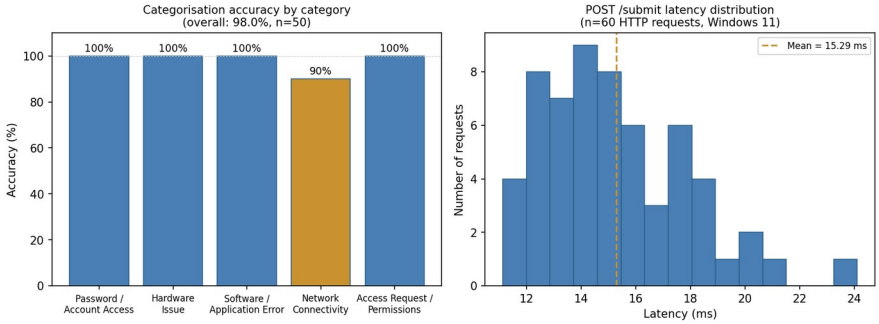


Figure 13. Per-category accuracy (left) and request latency distribution (right), both from measured data.

Because no separate user population was available to survey, the qualitative metric was a structured heuristic walkthrough of the running interface (Figures 2 to 6) against Nielsen's (1994) usability heuristics, using the six most applicable to a small internal tool. Five were fully met: visibility of system status (the result panel confirms the category and priority immediately), match between system and real world (the masking message is plain language), recognition rather than recall (the login page lists the two demo usernames), aesthetic and minimalist design (each page shows only what that step needs), and helping users recognise and recover from errors (an unknown username returns a specific instruction). Error prevention was only partly met: both forms use the browser's required attribute, which blocks an empty box, but a whitespace-only ticket passed that check and the server stored it. System testing found this (Section 4.4), and a server-side check now rejects it.

Table 6 breaks the accuracy result down by category, showing that the single error is isolated to one category pair and not spread across the test set.

Table 6

Categorisation Accuracy by Category

Category	Correct	Total	Accuracy
Password / Account Access	10	10	100.0%
Hardware Issue	10	10	100.0%
Software / Application Error	10	10	100.0%

Category	Correct	Total	Accuracy
Network Connectivity	9	10	90.0%
Access Request / Permissions	10	10	100.0%

4.4 System Testing

System testing aimed to verify the functionality, performance and stability of the integrated prototype. Four levels were run with pytest 9.1.1 on Windows 11 (Python 3.14.2), each test using its own temporary database. Unit tests (19) are white-box; integration tests (9) check routes, categorisation and database together; system tests (17) drive complete user journeys, error handling and basic security checks; and acceptance tests (7) verify the implemented parts of requirements FR1, FR4 and FR7 and the criteria in Table 3. The later levels test behaviour from outside, as black-box tests (Khan & Khan, 2012). Three further tests record known limitations as expected failures. Table 7 summarises the results, and Appendix C lists the test identifiers.

Table 7

Test Results by Level

Level	Tests	First run	After fixes
Unit	19	19 passed	19 passed
Integration	9	9 passed	9 passed
System	17	14 passed, 3 failed	17 passed
Acceptance	7	7 passed	7 passed
Known limitations	3	3 expected failures	3 expected failures
Total	**55**	**49 passed, 3 failed, 3 expected**	**52 passed, 3 expected**

Statement coverage was 98% overall (app.py 97%, categorization.py and models.py 100%). Figure 14 shows the first run, in which three system tests failed, and Figure 15 shows the same suite after the fixes.

```
PS C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype> python -m pytest -q --tb=line -p no:cacheprovider --cov=app --cov=categoriz
ation --cov=models --cov-report=term-missing --junitxml=evaluation\results\unit7_before_fix.xml
.................................xxx........FFF......                                                                                    [100%]
================================================================== FAILURES ==================================================================
E   assert (200 == 400)
    +  where 200 = <WrapperTestResponse streamed [200 OK]>.status_code
C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype\tests\test_system.py:80: assert (200 == 400)
E   assert 200 == 400
    +  where 200 = <WrapperTestResponse streamed [200 OK]>.status_code
C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype\tests\test_system.py:88: assert 200 == 400
E   assert [1, 2, 3] == [3, 2, 1]

      At index 0 diff: 1 != 3
      Use -v to get more diff
C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype\tests\test_system.py:99: assert [1, 2, 3] == [3, 2, 1]
============================================================== tests coverage ==============================================================
_____________________________________________ coverage: platform win32, python 3.14.2-final-0 _____________________________________________

Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
app.py                 65      2    97%   114-118
categorization.py      17      0   100%
models.py              27      0   100%
-------------------------------------------------
TOTAL                 109      2    98%
========================================================== short test summary info ===========================================================
FAILED tests/test_system.py::test_st09_empty_description_is_rejected - assert (200 == 400)
FAILED tests/test_system.py::test_st10_whitespace_only_description_is_rejected - assert 200 == 400
FAILED tests/test_system.py::test_st11_queue_lists_newest_ticket_first - assert [1, 2, 3] == [3, 2, 1]
3 failed, 49 passed, 3 xfailed in 2.71s
PS C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype>
```

Figure 14. First test run, with three failing system tests.

```
PS C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype> python -m pytest -q --tb=line -p no:cacheprovider --cov=app --cov=categoriz
ation --cov=models --cov-report=term-missing --junitxml=evaluation\results\unit7_after_fix.xml
.................................xxx.................                                                                                    [100%]
============================================================== tests coverage ==============================================================
_____________________________________________ coverage: platform win32, python 3.14.2-final-0 _____________________________________________

Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
app.py                 67      2    97%   122-126
categorization.py      17      0   100%
models.py              27      0   100%
-------------------------------------------------
TOTAL                 111      2    98%
52 passed, 3 xfailed in 2.48s
PS C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype>
```

Figure 15. Re-run after the fixes: 52 passed and 3 documented limitations.

The three failures exposed two defects (Table 8). A whitespace-only ticket passed the browser's required check and was stored as an empty ticket because the server did not validate it, and tickets created in the same second appeared in the wrong order because the queue sorted only by a one-second timestamp. Each was reproduced as a failing test first and then fixed, with a server-side check that returns HTTP 400 and a message, and the ticket id added as a sort tie-breaker. The three known limitations are the login without a password (KL-01), the keyword tie that misclassifies “My laptop shows offline” (KL-02), and the absence of CSRF protection (KL-03).

Table 8

Defects and Known Limitations Found by Testing

ID	Finding	Resolution
ST-09, ST-10	Empty or whitespace-only tickets were stored; the browser check stops only a blank box	Fixed: the server refuses them (HTTP 400) and shows a message
ST-11	Tickets created in the same second appeared in the wrong order	Fixed: ticket id added as a tie-breaker in the sort
KL-01	Login needs no password	Documented; real authentication planned
KL-02	A keyword tie sends “My laptop shows offline” to Hardware Issue	Documented; route ties to human review
KL-03	No CSRF protection on forms	Documented; add tokens
Stability	Response time degrades with ten simultaneous users	Documented; server database and production server

A stability test had ten simulated users each send 30 submissions at the same time, 300 in all (Figure 16). All succeeded, and the database held exactly 300 tickets, each with a distinct id, and 300 submission audit entries, so no rows were lost or duplicated. Speed did suffer: the mean response was 189.6 ms against 15.3 ms for a single user, the 95th percentile was 518.6 ms, and the slowest request took 4.7 s.

```
PS C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype> python evaluation\stability_test.py
Platform: Windows-11-10.0.26200-SP0 | Python 3.14.2

Simulated users: 10 at the same time, 30 tickets each
Requests sent:        300
Successful:           300
Failed:               0
Total time:           7.30 s   (41.1 successful requests per second)
Latency mean / p95 / max: 189.6 / 518.6 / 4703.0 ms
Tickets in database:  300   (distinct ids: 300)
Audit entries:        300 submissions, 10 logins
Data integrity:       OK
RESULT: PASS

Saved: C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype\evaluation\results\stability_results.json
PS C:\Users\saika\OneDrive\Desktop\msit5910-helpdesk-capstone\prototype>
```

Figure 16. Stability test: 300 submissions from 10 concurrent users, no failures.

As acceptance tester, the project owner ran six browser scenarios: login, a sensitive ticket, an ordinary ticket, a whitespace-only ticket, and the queue as agent and as identity manager (Figures 2 to 6, and Figure 17). All passed after the fixes.

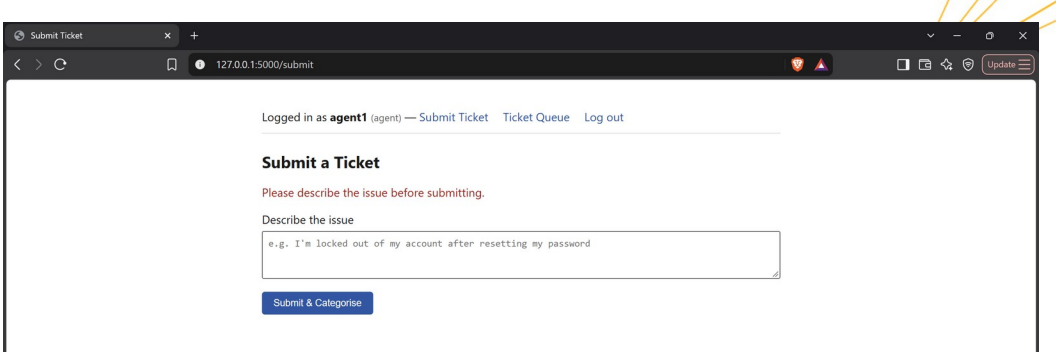


Figure 17. The message now shown for a whitespace-only ticket.

4.5 Summary of Results Against the Objectives

Table 9 relates the results to the five objectives set in Section 1.3. Objectives 1 to 4 were met within the stated scope. Objective 5 was met in part: version control is in use, with tagged releases, while deployment and maintenance exist as a documented plan, because containerisation, continuous integration and the other measures it describes have not yet been carried out.

Table 9

Results Against the Objectives

Objective	Outcome	Evidence
1. Standard categories and priorities	Met: five categories with default priorities were defined and implemented	Section 3.3; Table 6
2. Triage workflow and architecture	Met at design level; three of six modules were implemented	Chapter 3; Table 4
3. Working prototype of the core modules	Met: categorisation, role-based masking and the audit log work end to end	Section 4.2; Figures 3 to 6
4. Evaluation and testing	Met: 98.0% accuracy, 15.29 ms mean latency, 55 tests at 98% coverage, and a stability test	Sections 4.3 and 4.4
5. Deployment, maintenance and version control	Partly met: tagged releases in use; deployment and maintenance planned, not yet carried out	Section 5.4; Appendices B and D

5. DISCUSSION

5.1 Interpretation of Results

Table 5 and Figure 13 in Section 4.3 set out the evaluation metrics, and this section interprets them together with the testing in Section 4.4. They support a cautious claim: the implemented portion of the design works as intended, and the reason it works can be explained, not just observed. The 98% accuracy is high, but what matters more is that the 2% error traces back to one specific property of the rule-based design, tie-breaking by category order, and not to an opaque model making an unexplainable mistake. That matches the fairness and transparency argument made in Chapter 3 for a rule-based approach (NFR5), and here the explanation came from an actual misclassification found during testing.

The single-user latency (mean 15.29 ms) shows that the automated step is not a bottleneck, but the workload was small by design: one SQLite write and a keyword scan over five short category rows. The stability test adds the other half of the picture. With ten simultaneous users no request failed and no rows were lost, yet the mean rose to 189.6 ms and the slowest request took 4.7 seconds. Correctness therefore held under load while speed did not, which is consistent with SQLite allowing only one writer at a time (SQLite, n.d.) combined with a development web server. Section 5.4 sets out the response.

The per-category breakdown adds a detail the overall figure hides: four of five categories reached 100%, and the one shortfall sits in Network Connectivity. Because the error is explainable by a specific, reproducible mechanism, the more useful reading is that the design has a known weak point where two categories' keyword sets can both apply, not a general accuracy

ceiling. That points toward fixing the tie-breaking mechanism itself (Section 6.4) instead of expanding keyword lists.

System testing also changes how the usability result should be read. The one partial gap in the walkthrough, error prevention, proved to be a real defect and not only a design preference. The two defects found in Section 4.4 both sat between layers: the route accepted what the page should have prevented, and the query ignored timing. The unit tests and the earlier demonstration had missed them, which supports testing at several levels. A proof of concept can defer a polished validation layer, but it should not store empty tickets, and the fix was small.

5.2 Comparison with Literature Findings

The accuracy result is consistent with the central claim of the literature reviewed in Chapter 2: that simpler classification approaches can perform competitively with more complex ones on IT ticket text. Revina et al. (2020) found that explainable, linguistically-grounded features could match more complex classification approaches for IT ticket text, and Wahba et al. (2020) found that a simpler term-frequency representation performed comparably to static word-embedding models for the same task. A 98% accuracy figure from a plain keyword-matching function, achieved with no training data and no model tuning, is directionally consistent with that pattern: for a constrained, well-defined category set, sophistication is not obviously necessary to get most of the benefit. It is not a direct replication: this prototype uses hand-written keyword rules on a small synthetic test set, not the trained models those studies compared.

At the same time, Al-Hawari and Barham (2021), whose machine-learning-based help desk system is the main contrast point in the literature review, provide a reminder of what this implementation does not attempt. Their system was evaluated for its ability to generalise across a

much larger and more varied ticket population than the fifty-item test set used here, and a trained model would be expected to handle novel phrasing (vocabulary not anticipated by any keyword list) more gracefully than this project's fixed rule table could. The 2% error observed here is a design-level limitation (ties resolved by seed order), not a generalisation failure, but a larger, more adversarial test set would likely surface the rule-based approach's ceiling more clearly than the current one does.

Fuchs et al. (2022), in their broader literature review of machine-learning approaches to support ticket systems, note that reported accuracy figures across the field vary substantially by dataset, category granularity, and evaluation method, which is a useful caution against treating this project's 98% as directly comparable to any single published number; the test sets are not alike, and this project's figure should be read as evidence that the specific design works for its own evaluation scenarios, not as a benchmark claim against the wider field. Jäntti and Cater-Steel's (2017) process-management findings are reflected less in the accuracy number and more in the audit-log design described in Section 3.3 and shown in Figure 10: their action-research finding that consistent classification and clear process interfaces strengthen service-desk performance is the same argument this project makes for pairing categorisation with a recorded, reviewable audit trail instead of treating classification as an isolated technical problem.

The Security and Governance Layer was designed against ISO/IEC 27001:2022 and GDPR as reference points for information-security governance and data minimisation (International Organization for Standardization, 2022; European Union, 2016). The RBAC mechanism demonstrated in Figures 5 and 6 is a concrete instance of the least-privilege access-control principle both standards expect: a session holding the agent role is never sent security-sensitive ticket content, because the restriction is enforced in the /queue route itself, not in the interface.

The audit_log table similarly operationalises the accountability and traceability expectations common to both standards, recording who did what and when for every state-changing action. Neither standard is fully satisfied by this prototype. Encryption at rest, retention schedules, and data minimisation beyond “ask only for a description” are named in Chapter 3 as intended safeguards but are not implemented in the current codebase, and the demonstration login does not verify credentials (Section 5.3). The accurate claim is that the prototype demonstrates the access-control and audit-trail principles from these standards in working code, not that it is compliant with either standard as a whole.

5.3 Implications and Limitations

The clearest limitation is the gap between what Chapter 3 designed and what Chapter 4 actually implemented, made explicit in Table 4. FR6 (reporting on reassignment, backlog, and categorisation quality) was not built at all, and FR2, FR3, and FR5 were implemented in simplified form, most notably by collapsing impact and urgency into a single priority value instead of modelling them separately. This is a legitimate scope decision for a capstone-scale prototype, but it means the evaluation in this report is a test of the categorisation and access-control logic specifically, not of the full workflow described in Section 3.1.

A second limitation concerns the evaluation methods themselves. The accuracy test set, while written independently of the application's own keyword lists, was still constructed by the same person who built those keyword lists, which may share unconscious assumptions about how a ticket “should” be phrased; a test set written by someone unfamiliar with the implementation would be a stronger check. Similarly, heuristic evaluation is generally carried out by several independent evaluators, because a single evaluator tends to find only a minority of the usability

problems that a small panel would find; the qualitative result in this report is a single-evaluator walkthrough and should be read as a reasonable first pass, not a comprehensive usability finding.

Third, the system's current configuration limits how far the performance results can be generalised. The Flask development server is explicitly not intended for production use, SQLite does not handle concurrent writes as gracefully as a server-based database under real multi-user load, and the secret key is hardcoded for local demonstration and not supplied through the environment variables recommended in Section 5.4. None of this affects the correctness of the measurements, since all 60 latency requests and all 50 accuracy checks were actually run, but it means the numbers should not be read as representative of a deployed multi-agent system or of other hardware: every request came from a single Windows 11 machine, and the stability test showed response times degrading under ten simultaneous users.

Fourth, authentication is weaker than authorisation. The demonstration login accepts a username without a password, and the login page lists the two valid usernames, so the RBAC behaviour in Figures 5 and 6 shows that the masking rule is enforced correctly once a role has been assigned, not that a user is prevented from simply selecting the privileged role. Credential checks and multi-factor authentication would be needed before the access-control claim could be made for a real deployment. System testing recorded this and two further gaps, the absence of CSRF protection and the keyword tie, as expected failures (Table 8). The OWASP Foundation (2021) places broken access control, security misconfiguration and identification and authentication failures among its top risks, and the prototype's hardcoded key, debug-mode server and password-less login touch all three.

Finally, keyword-based tie-breaking by seed order is a structural property of the current design and not a bug confined to one keyword. Any two categories that match the same number of

keywords in a ticket will always resolve the same way, whichever interpretation is more appropriate. One instance was found and fixed during unit testing and a second was observed in the accuracy test and left unfixed, which is itself a useful finding: the underlying mechanism is the more durable thing to address.

5.4 Deployment, Configuration and Maintenance

Three deployment options were weighed: an on-premise server, a managed cloud platform and a Docker container (Appendix B). The Docker container was selected, supported by a recent literature-based preprint reporting that containers are generally associated with shorter startup times and near-native execution for many workloads, though the benefit depends on the workload (Gurramkonda et al., 2026). A container image fixes the Python version and dependencies, so the categorisation behaviour measured here would be reproducible wherever the image runs, although absolute response times such as the 15.29 ms mean would still depend on the host hardware.

The configuration plan addresses a gap visible in the code: app.py hardcodes its Flask secret key, with a comment acknowledging this is fine for a local demonstration but not beyond it. The plan moves the key, the debug flag and the database path into environment variables, following the Config principle of the twelve-factor app, which holds that configuration should be strictly separated from code so that the same build can run unmodified across environments (Wiggins, n.d.).

The maintenance plan uses the four types of maintenance defined in ISO/IEC/IEEE (2022). Corrective work repairs defects such as the two fixed in Section 4.4; adaptive work follows changes to Python, Flask or container images; perfective work adds the unbuilt requirements and

real authentication; and preventive work covers pinned dependencies, automated tests and backups. Releases follow semantic versioning (Preston-Werner, n.d.), each bug is reproduced as a failing test before it is fixed, and continuous integration, reported to work on maintenance projects (Shahin et al., 2017), would run the 55 tests on every push. A quarterly cycle re-runs the accuracy set and the stability test, and the schedule is in Appendix B. Table 10 lists the main risks and their mitigations.

Table 10

Maintenance Risks and Responses

Risk	Evidence	Mitigation and contingency
Security weaknesses	Hardcoded key, no password, no CSRF, debug server; OWASP Foundation (2021) A01, A05, A07	Secrets in environment variables, hashed passwords with MFA, CSRF tokens, production server; after a breach, rotate keys and redeploy the last good tag
Data loss or inconsistency	SQLite allows one writer at a time (SQLite, n.d.)	Nightly backups with restore tests; move to a server database beyond about ten active agents; restore the last backup if needed
Slow response	Slowest request 4.7 s with ten users	Monitor the 95th percentile; quarterly load test; production server
Misclassification	KL-02; new phrasing	Review the human-review queue weekly; add keywords with regression tests; alert if accuracy on the test set falls below 90%
Outdated components	OWASP Foundation (2021) A06	Pin versions; test monthly updates through continuous integration
Usability problems	Whitespace tickets reached the server	Collect agent feedback; validate every input on the server

5.5 Threats to Validity

Four threats limit how far the results can be generalised. Internal validity is affected by authorship: one person wrote the code, the keyword lists, the test set and the tests, so shared assumptions are possible. The test tickets were written independently of the keyword lists and each defect was reproduced as a failing test before it was fixed, which reduces but does not

remove the risk. Construct validity is limited because 50 hypothetical tickets measure how clear the categories are, not how much triage effort a real helpdesk would save, and a response time measures the software and not the human process around it.

External validity is limited by the environment: all measurements came from one Windows 11 machine with the Flask development server and a single SQLite file, and the stability test used only ten simulated users. Finally, latency varies from run to run, so the reported means are representative figures and not exact constants; the 60-request and 300-request samples are large enough to show the pattern, but not to support fine comparisons. None of these threats changes the main findings, but each argues for the independent test set, second tester and larger load test proposed in Section 6.4.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of Findings

This capstone set out to show that a simplified ticket categorisation model and a structured triage workflow could make IT helpdesk routing clearer and more consistent, without becoming a full software development effort. Chapters 1 to 3 argued this from the literature and design reasoning. The implementation and testing then asked a narrower, concrete question: can the core logic of that design, automatic categorisation and role-based access to sensitive ticket detail, actually be built, tested and shown to work? On the evidence in this report the answer is yes, within the scope deliberately chosen. A working Flask application categorised 49 of 50 independently written test tickets correctly, responded in about 15 milliseconds per request for a single user, enforced its access-control rule on every request tested, and passed 52 of 55 automated tests, the other three being documented limitations, with 98% statement coverage. Under ten simultaneous users it lost no records but slowed noticeably.

The project remains small by design, and this report has tried throughout not to claim otherwise: three of six modules were implemented, six of seven functional requirements were only partly realised and the seventh, reporting, was not built, the test set held 50 tickets, and the usability walkthrough had a single evaluator. Within those limits the evidence is solid: defects that were found and fixed, performance figures with timed requests behind them, and an access-control rule that held on every request it was tested against.

6.2 Contributions

The main contribution is the evidence trail behind intentionally simple categorisation logic: defects found by testing and not by inspection, then fixed and verified; performance claims backed by timed requests instead of estimates; a requirements table that reports partial and missing implementation openly; and a maintenance plan with a risk register. That trail is itself a demonstration of the fairness-through-transparency argument of the design. For the field, the contribution is modest but concrete: a small, reproducible example, with code, tests and measurements published in a versioned repository (Ganguly, 2026), of the simple, explainable approach that Revina et al. (2020) and Wahba et al. (2020) found competitive on IT ticket text, applied to helpdesk tickets with access control for sensitive categories built in.

6.3 Lessons Learned and Challenges

Testing and measurement change what a developer can credibly claim. Before the unit tests existed, the reset-keyword issue was invisible; before the system tests, the empty-ticket and queue-order defects were invisible; before the HTTP-level evaluation, “the system is fast enough” was an assumption, and afterwards it was a number with a method attached. The recurring tie-breaking misclassification reinforced the point that a passing suite does not mean every related issue has been found.

A second lesson is that evidence is only as reliable as the environment it comes from. Re-running the measurements on the real application, with its original pages, corrected an earlier usability finding that had assumed the forms had no client-side validation. The main challenges were keeping the implementation deliberately small while still demonstrating the design, and being exact about which requirements were met, partly met or deferred.

6.4 Future Work

Six development directions follow from the limitations. First, revisit the tie-breaking mechanism, for example by scoring ties on keyword specificity or by routing true ties to the human-review fallback already used for zero-match descriptions. Second, build the deferred requirements (FR2, FR3, FR5 and especially the FR6 reporting module), together with the guided prompts of FR1 and the assignment and escalation audit of FR7, on the audit_log table that is already being populated. Third, add real authentication with hashed passwords and multi-factor authentication, CSRF protection and secrets held in environment variables, which addresses KL-01 and KL-03; the tie-handling change in the first direction addresses KL-02. Fourth, carry out the deployment and maintenance plan: containerise the application, replace the development server, move to a server database before concurrent use grows, add continuous integration, and repeat the stability test. Fifth, strengthen the evaluation with a larger test set written by someone other than the implementer, a second tester and a multi-evaluator usability study. Sixth, close the FR4 gap: a sensitive ticket is flagged and restricted, but nothing notifies the identity management role that one is waiting, so the immediate escalation path is an access restriction today and not a notification.

Beyond development, one research recommendation follows from the literature review: running the same 50 tickets through a trained classifier of the kind Al-Hawari and Barham (2021) describe would test directly whether the rule-based approach keeps its explainability advantage without a large cost in accuracy.

UNIVERSITY OF THE PEOPLE

REFERENCES

Al-Hawari, F., & Barham, H. (2021). A machine learning based help desk system for IT service management. *Journal of King Saud University – Computer and Information Sciences, 33*(6), 702–718. https://doi.org/10.1016/j.jksuci.2019.04.001

AXELOS. (2019). *ITIL foundation: ITIL 4 edition*. TSO.

European Union. (2016). *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation)*. https://eur-lex.europa.eu/eli/reg/2016/679/oj

Fuchs, S., Drieschner, C., & Wittges, H. (2022). Improving support ticket systems using machine learning: A literature review. In *Proceedings of the 55th Hawaii International Conference on System Sciences* (pp. 1893–1902). https://hdl.handle.net/10125/79570

Ganguly, S. S. (2026). *msit5910-helpdesk-capstone* (Version 1.2.0) [Source code]. GitHub. https://github.com/saikatsunilganguly-cpu/msit5910-helpdesk-capstone

Gurramkonda, F., Malla, A., Tamboli, S., & Nazneen, S. (2026). Docker containers vs. virtual machines: A comparative study of architecture, performance, configuration, and security. *arXiv*. https://doi.org/10.48550/arXiv.2609.16148

International Organization for Standardization. (2022). *ISO/IEC 27001:2022 information security, cybersecurity and privacy protection—Information security management systems—Requirements*. https://www.iso.org/standard/27001.html

ISO/IEC/IEEE. (2022). *ISO/IEC/IEEE 14764:2022: Software engineering—Software life cycle processes—Maintenance*. https://www.iso.org/standard/80710.html

Jäntti, M., & Cater-Steel, A. (2017). Proactive management of IT operations to improve IT services. *Journal of Information Systems and Technology Management, 14*(2), 191–218. https://doi.org/10.4301/S1807-17752017000200004

Khan, M. E., & Khan, F. (2012). A comparative study of white box, black box and grey box testing techniques. *International Journal of Advanced Computer Science and Applications, 3*(6), 12–15. https://doi.org/10.14569/IJACSA.2012.030603

Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. In *CHI '94: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 152–158). ACM. https://doi.org/10.1145/191666.191729

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/2021/

Preston-Werner, T. (n.d.). *Semantic versioning 2.0.0*. Retrieved October 15, 2026, from https://semver.org/

Revina, A., Buza, K., & Meister, V. G. (2020). IT ticket classification: The simpler, the better. *IEEE Access, 8*, 193380–193395. https://doi.org/10.1109/ACCESS.2020.3032840

Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access, 5*, 3909–3943. https://doi.org/10.1109/ACCESS.2017.2685629

SQLite. (n.d.). *Appropriate uses for SQLite*. Retrieved October 15, 2026, from https://www.sqlite.org/whentouse.html

Wahba, Y., Madhavji, N. H., & Steinbacher, J. (2020). Evaluating the effectiveness of static word embeddings on the classification of IT support tickets. In *CASCON '20: Proceedings of the 30th Annual International Conference on Computer Science and*

Software Engineering (pp. 198–206). IBM Corp. https://doi.org/10.5555/3432601.3432628

Wiggins, A. (n.d.). *The twelve-factor app*. Retrieved October 15, 2026, from https://12factor.net/

UNIVERSITY OF THE PEOPLE

APPENDICES

Appendix A. Functional and Non-Functional Requirements

The requirements below were defined during the design phase and underpin the evaluation in Chapter 4. Implementation status is given in Table 4.

Table 11

Functional Requirements

ID	Functional requirement
FR1	The system shall provide guided, plain-language prompts that allow users to select one of at least five standard ticket categories.
FR2	The system shall capture a structured ticket record containing category, impact, urgency, relevant description, and submission timestamp.
FR3	The system shall apply a standard triage workflow that routes tickets using category, priority, impact, and urgency.
FR4	The system shall flag security-sensitive or high-impact tickets for an immediate escalation path.
FR5	The system shall record ticket category, priority, assignment, status changes, and resolution information for analysis.
FR6	The system shall produce reports on reassignment rate, backlog size, resolution-time trends, and categorisation quality.
FR7	The system shall enforce role-based access control and maintain an audit trail for ticket assignment and escalation changes.

Table 12

Non-Functional Requirements

ID	Non-functional requirement	Design implication
NFR1	Usability: non-technical users should be able to submit and categorise a ticket without specialist knowledge.	Plain-language prompts, minimal fields, and guided choices.
NFR2	Reliability: workflow decisions should be applied consistently for equivalent tickets.	Standard categories, a priority matrix, and defined routing rules.
NFR3	Scalability: additional categories or queues should be added without redesigning the workflow.	Configurable category and routing lookup tables.
NFR4	Security: ticket data should be protected from unauthorised access or inappropriate disclosure.	RBAC, encryption, audit logging, retention controls, and least privilege.

ID	Non-functional requirement	Design implication
NFR5	Transparency and fairness: decision prompts should be understandable and applied consistently.	Explainable category prompts and documented escalation rules.
NFR6	Performance: the process should reduce avoidable initial-triage effort.	Fewer unclear choices and predefined routing paths.

NFR4 and the data-minimisation practices in the Ticket Submission Interface reflect the principles of data minimisation, purpose limitation and secure processing in the General Data Protection Regulation (European Union, 2016).

Appendix B. Deployment, Configuration and Maintenance Plan

Deployment method. An on-premise server would give full control but adds infrastructure overhead disproportionate to a capstone-scale prototype, and a managed cloud platform would tie the deployment to one provider's console and billing. A Docker container was selected because it captures the exact Python version, dependencies and file layout the Flask application needs, so it runs the same way on a laptop or a cloud container service.

Configuration management. Configuration management contributes to reliability by keeping anything that varies between environments (secrets, file paths, debug mode) out of the code. The prototype hardcodes its Flask secret key, marked in the code as fine for a local demonstration; a real deployment would move it, with the debug flag and database path, into environment variables (Wiggins, n.d.). This also prevents a secret key from being committed to the public repository by accident.

Table 13

Configuration and Setup Requirements

Item	Detail
Prerequisites	Any operating system that runs Python 3.12 or later (measurements taken on Windows 11, Python 3.14.2); Flask, pytest and pytest-cov from requirements.txt; no external database server, because SQLite is bundled with Python
Environment variables (recommended, not yet implemented)	FLASK_SECRET_KEY replaces the hardcoded key; FLASK_DEBUG=0 disables debug mode, which exposes a code-execution console on error pages; DATABASE_PATH sets where helpdesk.db is written, so a container can use a persistent volume
Local install and run	pip install -r requirements.txt, then python app.py
Container build and run	docker build -t helpdesk-prototype . and docker run -p 5000:5000 -e FLASK_SECRET_KEY=... helpdesk-prototype
Production-style run	gunicorn --bind 0.0.0.0:5000 app:app, after adding gunicorn to requirements.txt, in place of the Flask development server

Table 14

Proposed Maintenance Schedule

Frequency	Activities
Every change	Run all 55 tests; update documentation and release notes
Weekly	Review the audit log and the tickets sent to human review
Monthly	Check dependency and security updates; test a database restore
Quarterly	Re-run the 50-ticket accuracy set and the stability test; review keywords and the OWASP list
After an incident	Find the cause, add a regression test, release a patch

Appendix C. Test Catalogue

Table 15

Test Identifiers and Objectives

Level	Identifiers	What the tests check
Unit	19 tests	Category matching, fallback, case-insensitivity and the tie-break regression; database creation, seeding, idempotency and audit-log insertion
Integration	IT-01 to IT-09	Stored category, priority and flag; audit entries; session role; persistence between requests; submitted-by

Level	Identifiers	What the tests check
System	ST-01 to ST-17	Login protection, unknown user, masking for agent and identity manager, logout, empty and whitespace tickets, queue order, script escaping, SQL-injection text, long and Unicode text, malformed requests
Acceptance	AT-01 to AT-07	FR1, FR4 and FR7; at least 90% accuracy on the 50-ticket set; median response under 100 ms; uncertain tickets sent to a human
Known limitations	KL-01 to KL-03	Password check, keyword tie and CSRF, recorded as expected failures
Stability	1 script	Ten simultaneous users, 30 tickets each: failures, latency and data integrity

Appendix D. Repository and Release History

The project repository is available at https://github.com/saikatsunilganguly-cpu/msit5910-helpdesk-capstone. Table 16 lists the tagged releases, and Figure 18 shows the latest release page.

Table 16

Tagged Releases

Tag	Contents	Phase
v0.1.0	First working prototype: categorisation and role-based access logic	Prototype
v0.2.0	Reset-keyword fix and the 19-test unit suite	Unit testing
v1.0.0	Release of the tested prototype	Unit testing
v1.1.0	Page templates, 50-ticket test set, evaluation scripts and Windows results	Evaluation
v1.2.0	Integration, system and acceptance tests, two bug fixes and the stability test	System testing



Figure 18. Release v1.2.0 on GitHub.